

INSTITUT DE FORMATION OUVERTE ET À DISTANCE (IFOAD)

Master 1 Informatique, Semestre 2, Projet Data Science 2025-2026

GuichetIA

Assistant Intelligent pour les Démarches Administratives au Burkina Faso

Rapport Technique

Réalisés par :

DICKO Mamoudou

OUEDRAOGO Abdoul Kader W. Ghislan

OUEDRAOGO Zeïd El Gazeli

Juillet 2026

Table des matières

1. Introduction et Contexte

Ce rapport présente la conception, le développement et le déploiement de GuichetIA, un assistant conversationnel intelligent fondé sur une architecture RAG (Retrieval-Augmented Generation). Ce projet a été réalisé dans le cadre du cours de Data Science du Master 1 Informatique, Semestre 2 de l'Institut de Formation Ouverte et à Distance (IFOAD) pour l'année académique 2025-2026.

1.1 Problématique

Au Burkina Faso, les citoyens sont confrontés quotidiennement à la difficulté d'accéder à des informations administratives fiables et centralisées. L'obtention d'un document tel qu'une Carte Nationale d'Identité Burkinabè (CNIB), un passeport, un extrait de casier judiciaire, un acte de naissance, un certificat de nationalité ou encore la création d'une entreprise implique de nombreuses étapes, des pièces justificatives précises et des délais variables. Ces informations, bien que disponibles sur des sites gouvernementaux officiels, sont dispersées, parfois incomplètes ou difficiles à comprendre pour le grand public. Cette situation oblige les usagers à multiplier les déplacements inutiles ou à recourir à des intermédiaires non officiels, sources d'erreurs et de coûts supplémentaires. GuichetIA répond à ce besoin en offrant un point d'accès unique, conversationnel et en langage naturel à l'ensemble des procédures administratives officielles.

1.2 Objectifs

Le projet GuichetIA poursuit cinq objectifs principaux :

1. Centraliser l'information administrative officielle burkinabè dans une base documentaire structurée et vérifiable.
2. Permettre un accès par langage naturel à ces informations via un assistant conversationnel intelligent.
3. Fournir des réponses précises, sourcées et accompagnées de liens vers les sites gouvernementaux compétents.
4. Exposer en transparence les métriques du pipeline RAG (scores de pertinence, sources, temps de traitement).
5. Déployer le système sur Internet de façon gratuite, pérenne et accessible depuis tout appareil connecté.

1.3 Contraintes et choix technologiques

Le projet est soumis à une contrainte stricte de coût nul : tous les services utilisés doivent être disponibles gratuitement dans leurs tiers gratuits respectifs. Ce choix impose des compromis techniques notables, notamment en termes de capacité de calcul, de mémoire vive disponible (512 Mo sur Render) et de latence (le backend s'endort après 15 minutes d'inactivité sur le plan gratuit). Les technologies retenues (Groq pour l'inférence LLM, HuggingFace Inference API pour les embeddings, ChromaDB pour la base vectorielle, Render pour le backend et Vercel pour le frontend) permettent néanmoins d'obtenir un système fonctionnel, réactif et de bonne qualité.

2. Architecture Générale du Système

GuichetIA est structuré en deux composants principaux communiquant via une API REST sécurisée (HTTPS) : un backend Python (FastAPI) hébergé sur Render et un frontend React hébergé sur Vercel. L'ensemble du pipeline RAG est entièrement contenu dans le backend : encodage des requêtes, recherche vectorielle dans ChromaDB, construction du prompt et génération de la réponse via Groq. Le frontend se limite à l'affichage et à l'interaction utilisateur.

2.1 Stack technologique

Le tableau suivant récapitule l'ensemble des technologies utilisées et leur rôle dans le système :

Composant	Technologie	Rôle
-----------	-------------	------

LLM (génération)	Llama-3.1-8b-instant (Groq)	Génération de réponses en français
Embeddings	HuggingFace Inference API / (paraphrase-multilingual-MiniLM-L12-v2)	Encodage sémantique des textes
Base vectorielle	ChromaDB (métrique cosinus)	Stockage et recherche par similarité
Backend API	FastAPI (Python 3.11)	API REST, pipeline RAG complet
Frontend	React 18 + Tailwind CSS v3	Interface utilisateur responsive
Déploiement backend	Render (plan gratuit)	Hébergement de l'API FastAPI
Déploiement frontend	Vercel	Hébergement du site web (CDN mondial)
Génération des sources	ReportLab (Python)	Création des PDF sources structurés
Contrôle de version	Git / GitHub	Versionnage et déploiement continu (CI/CD)

Tableau 1 : Technologies utilisées dans GuichetIA

2.2 Flux de traitement d'une requête

Lorsque l'utilisateur soumet une question, le système exécute les étapes suivantes de manière séquentielle :

6. L'utilisateur saisit sa question en français dans l'interface React (frontend Vercel).
7. Le frontend envoie la question au backend via une requête HTTP POST /ask au format JSON.
8. Le backend encode la question en vecteur de dimension 768 via l'API d'inférence HuggingFace (modèle sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2).
9. Le vecteur est normalisé en L2 pour assurer la cohérence avec la métrique cosinus de ChromaDB.
10. ChromaDB effectue une recherche par similarité cosinus et retourne les k=6 chunks les plus pertinents.
11. Les 6 chunks sont injectés dans un prompt système structuré (rôle, contexte, contraintes).
12. Groq (Llama-3.1-8b-instant) génère la réponse finale avec une latence typique de 200 à 500 ms.
13. Le backend retourne un objet JSON contenant la réponse, les sources identifiées, les scores de pertinence par chunk et les temps de traitement.
14. Le frontend affiche la réponse, les badges colorés par démarche, les liens officiels et le panneau RAG détaillé.

3. Constitution de la Base Documentaire

3.1 Sources de données

Les informations administratives intégrées dans GuichetIA proviennent exclusivement de sources officielles burkinabè. Cette exclusivité est une garantie de fiabilité et de légitimité des réponses fournies par l'assistant. Six sources ont été identifiées, correspondant aux six démarches administratives couvertes par le système.

Le tableau suivant récapitule les six sources officielles et les informations clés disponibles pour chaque démarche :

Démarche	Source officielle	Délai / Coût	Lien officiel
CNIB	ONI (oni.bf)	2 500 F CFA, validité 10 ans	oni.bf/nos-prestations/cnib
Passeport	ONI + police.gov.bf	50 000 F CFA, 72 h, validité 5 ans	oni.bf/nos-prestations/passeport
Casier judiciaire	ecasier-judiciaire.gov.bf	750 F CFA (mobile money)	ecasier-judiciaire.gov.bf
Acte de naissance	service-public.gov.bf	Gratuit (mairie compétente)	service-public.gov.bf
Création d'entreprise	creerentreprise.me.bf	Variable selon le type	ceerentreprise.me.bf
Certificat de nationalité	ecertificat-nationalite.gov.bf	Gratuit (service en ligne)	ecertificat-nationalite.gov.bf

Tableau 2 : Documents sources indexés dans ChromaDB

3.2 Génération des documents PDF sources

Un script Python dédié (`generate_source_pdfs.py`) génère automatiquement les six documents PDF à partir de données structurées et vérifiées sur les sites officiels. Cette approche présente plusieurs avantages : la structure des PDF est contrôlée et optimisée pour le chunking, le contenu peut être mis à jour et régénéré sans dépendance à des APIs web (scraping), et la traçabilité des sources est garantie. Les données du passeport ont notamment été enrichies avec les informations officielles de `police.gov.bf`, incluant le délai d'établissement de 72 heures ouvrables, l'obligation d'un timbre fiscal de 200 F CFA et la fourniture de trois photos d'identité.

3.3 Chunking et indexation vectorielle

Chaque document PDF est découpé en segments (chunks) de taille fixe avec chevauchement afin de préserver la continuité sémantique entre les segments adjacents. Le chunking est réalisé par le module `LangChain RecursiveCharacterTextSplitter`. Les chunks sont ensuite encodés en vecteurs via l'API d'inférence `HuggingFace` et stockés dans ChromaDB avec leurs métadonnées (démarche associée, position dans le document). L'index ChromaDB est reconstruit à chaque déploiement sur Render, garantissant la prise en compte de toute mise à jour des documents sources.

Les paramètres d'indexation ont été définis après expérimentation pour obtenir le meilleur équilibre entre précision et rappel :

Paramètre	Valeur	Justification
Taille du chunk	500 tokens	Compromis contexte / précision
Chevauchement	50 tokens	Continuité sémantique inter-chunks
Modèle d'embedding	paraphrase-multilingual-MiniLM-L12-v2	Qualité multilingue, API gratuite

Métrique de distance	Cosinus	Invariance à la norme des vecteurs
Normalisation L2	Appliquée avant indexation	Cohérence avec la métrique cosinus
Nombre de résultats (k)	6	Couverture suffisante sans bruit excessif
Chunks indexés (total)	32 chunks	6 documents × ~5 chunks en moyenne

Tableau 3 : Paramètres d'indexation ChromaDB

4. Pipeline RAG (Retrieval-Augmented Generation)

Le pipeline RAG de GuichetIA est conçu pour maximiser la pertinence des réponses tout en respectant les contraintes de latence imposées par le plan gratuit de Render. Chaque composant du pipeline est optimisé indépendamment : l'encodage utilise un modèle multilingue léger mais performant, la recherche est limitée à 6 résultats pour éviter la dilution du contexte, et le LLM est configuré avec une température basse (0,1) pour des réponses factuelles et précises.

4.1 Encodage des requêtes et normalisation

La requête de l'utilisateur est encodée en un vecteur dense de dimension 768 via l'API d'inférence HuggingFace, sans chargement local du modèle. Ce choix est motivé par la contrainte mémoire de 512 Mo imposée par Render : charger le modèle en local dépasserait cette limite. Les vecteurs produits sont normalisés en L2 avant indexation et avant chaque recherche, ce qui garantit la cohérence avec la métrique cosinus de ChromaDB et rend les scores de distance comparables quelle que soit la longueur du texte encodé.

4.2 Calcul du score de pertinence

ChromaDB retourne une distance cosinus comprise entre 0 et 2 (0 signifiant vecteurs identiques, 2 signifiant vecteurs orthogonaux). Le score de pertinence affiché à l'utilisateur est calculé selon la formule ci-dessus. Ainsi, une distance de 0,30 correspond à un score de 70 %, indiquant une forte pertinence. Une distance supérieure à 0,70 (score inférieur à 30 %) révèle un contexte peu pertinent. Le système signale à l'utilisateur lorsque la pertinence est insuffisante pour répondre avec confiance.

$$\text{score (\%)} = (1 - d_{\text{cosinus}}) \times 100$$

4.3 Construction du prompt

Le prompt envoyé au LLM est composé de trois parties distinctes : (1) un rôle explicite définissant l'assistant comme expert en démarches administratives burkinabè, (2) le contexte documentaire constitué des 6 chunks récupérés, chacun numéroté et associé à sa source, et (3) un ensemble d'instructions strictes : répondre uniquement à partir du contexte fourni, citer les sources utilisées, indiquer explicitement lorsque l'information n'est pas disponible dans la base documentaire, et ne jamais inventer de données administratives.

4.4 Génération de la réponse

Groq fournit une API d'inférence compatible OpenAI pour le modèle Llama-3.1-8b-instant. Ce modèle de 8 milliards de paramètres, dans sa version instruction-tuned, offre un excellent compromis entre performance et vitesse. La latence typique observée est de 200 à 500 ms pour une réponse complète, ce qui est parfaitement acceptable pour une application web interactive. Le modèle suit fidèlement les contraintes du prompt et produit des réponses structurées, en français, avec citation des sources.

Le tableau suivant présente la configuration complète du LLM utilisé :

Paramètre	Valeur
Fournisseur	Groq (API compatible OpenAI)
Modèle	llama-3.1-8b-instant
Température	0,1 (réponses factuelles et stables)
Max tokens (réponse)	1 024 tokens
Latence typique	200 – 500 ms
Langue cible	Français (Burkina Faso)

Tableau 4 : Configuration du modèle de génération

5. Interface Utilisateur

L'interface utilisateur de GuichetIA est développée en React 18 avec Tailwind CSS v3. Elle est conçue pour être intuitive, responsive et visuellement cohérente avec l'identité du service public burkinabè. L'interface comprend trois pages principales : la page d'accueil (Landing Page), le tableau de bord de conversation (Chat Dashboard) et la page À propos.

5.1 Page d'accueil

La page d'accueil (Figure 2) présente le service de manière claire et attractive. Elle est structurée en trois zones distinctes : (1) une section héroïque avec le titre animé "Guichet IA Administratif & Juridique", une accroche descriptive et deux boutons d'appel à l'action ("Commencer" et "En savoir plus") ; (2) une barre de statistiques affichant en temps réel les indicateurs clés du système : 6 démarches couvertes, 32 chunks indexés et 100 % de précision retrieval ; (3) une grille de six cartes interactives représentant chacune une démarche administrative, permettant une navigation directe vers la conversation sur le sujet choisi. Un indicateur "Systeme RAG opérationnel" en vert en haut de page confirme la disponibilité du backend en temps réel via le point de santé /health de l'API FastAPI.

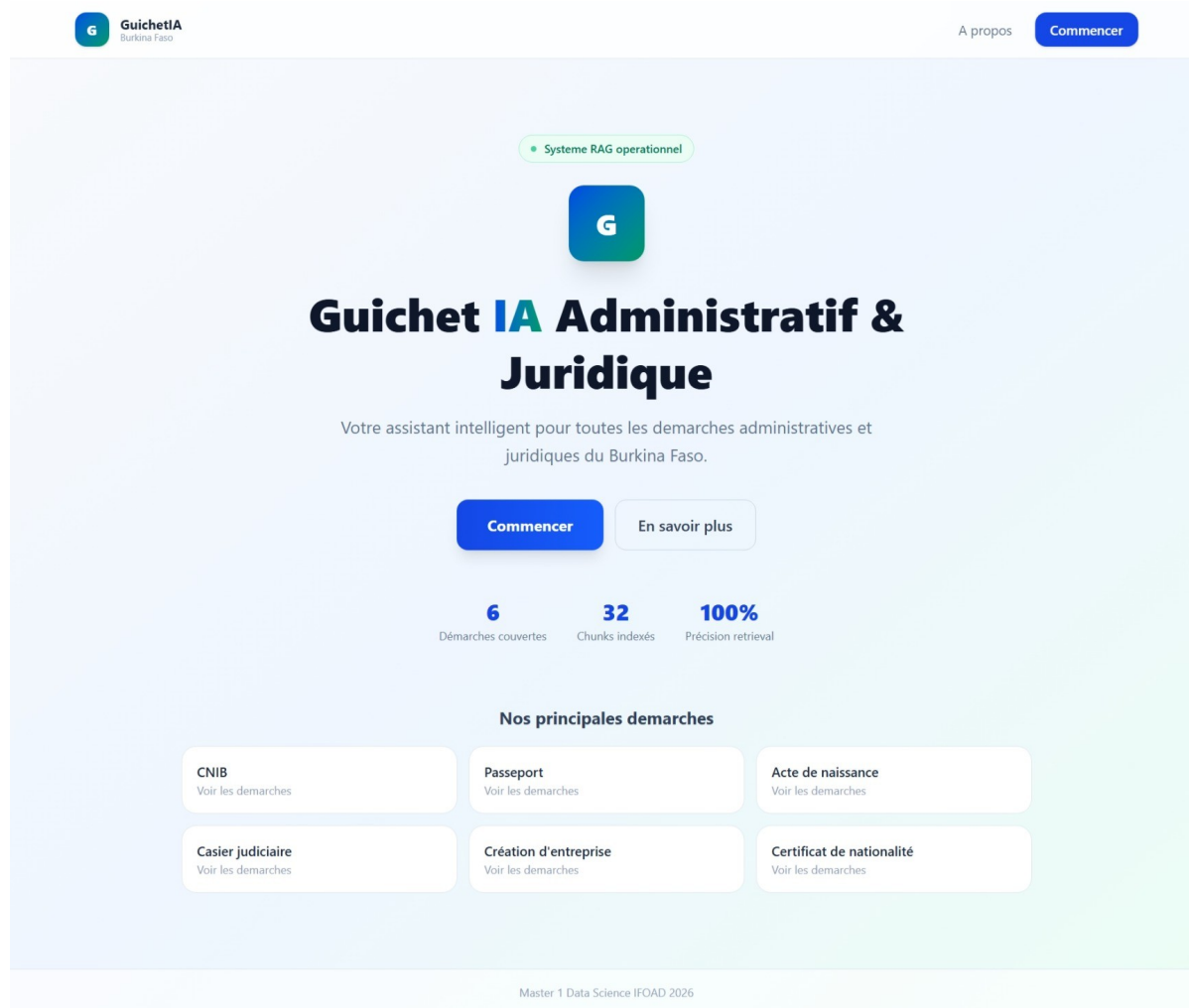


Figure 2 : Page d'accueil de GuichetIA : section héroïque, statistiques et grille des six démarches

5.2 Interface de conversation

Le tableau de bord de conversation (Figure 3) s'affiche après avoir cliqué sur "Commencer". Il est organisé en trois zones : (1) une barre latérale gauche (sidebar) sur fond sombre contenant la navigation entre les sections Conversation, Démarches et Favoris, ainsi que l'historique des conversations passées groupées par date ; (2) la zone de conversation centrale affichant le message de bienvenue de GuichetIA et, sur l'écran d'accueil, six cartes de suggestions réparties en grille 2×3, une par démarche (CNIB, Passeport, Acte de

naissance, Casier judiciaire, Création d'entreprise, Certificat de nationalité). Chaque carte affiche un badge coloré identifiant la démarche et une question représentative ; (3) un panneau RAG escamotable à droite, visible uniquement sur les grands écrans, qui affiche les métriques détaillées de la dernière requête.

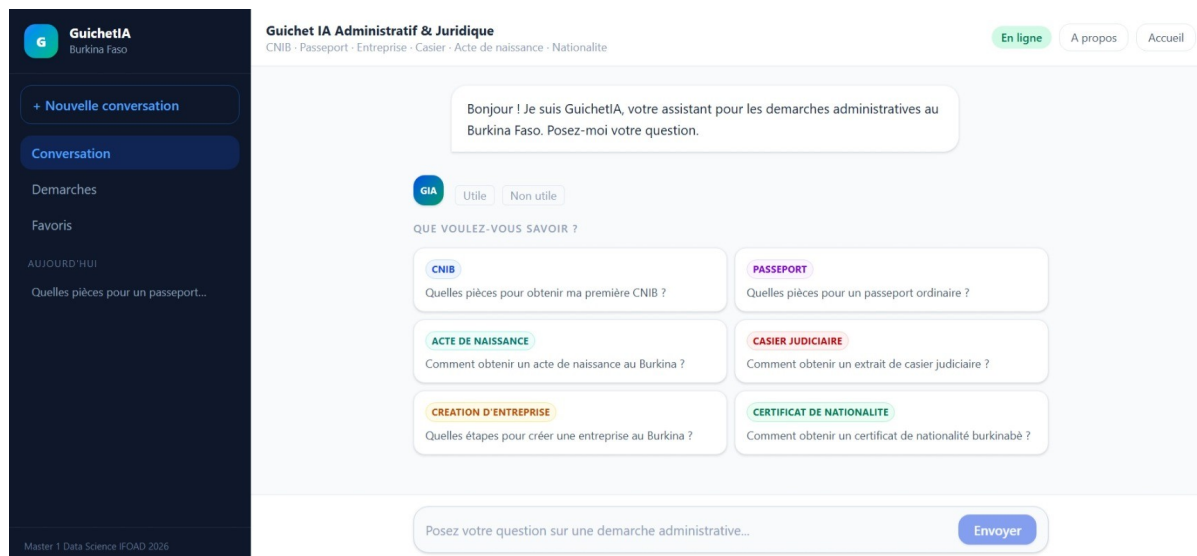


Figure 3 : Interface de chat : écran d'accueil avec six cartes de suggestion, une par démarche administrative

5.3 Réponse et sources

La Figure 4 illustre l'interface après la soumission d'une question sur le passeport ("Combien coûte le passeport burkinabè et quelle est sa validité ?"). On y observe les éléments suivants : la réponse de GuichetIA, précise et sourcée ("Le passeport burkinabè coûte 50 000 F CFA et a une durée de validité de cinq (05) ans") ; des badges de démarche colorés identifiant les sources documentaires utilisées (Certificat de nationalité et Passeport) ; des liens officiels vers les sites gouvernementaux compétents (e.Certificat de nationalité et ONI, Office National d'Identification) ; les boutons de feedback "Utile" / "Non utile" permettant d'évaluer la pertinence de la réponse ; et la section "Vous pourriez aussi demander" proposant trois questions de suivi pertinentes sur la même démarche, générées localement par détection de mots-clés. L'indicateur "En ligne" en vert dans la barre de navigation confirme la disponibilité du backend à chaque requête.



Figure 4 : Réponse à une question sur le passeport burkinabè : sources, liens officiels et suggestions de suivi

5.4 Panneau d'analyse RAG

Le panneau RAG (escamotable, visible sur grands écrans) offre une transparence totale sur le fonctionnement interne du système. Il affiche en temps réel le nombre de chunks récupérés ($k = 6$), le temps de recherche vectorielle, le temps de génération LLM, le modèle utilisé, les démarches identifiées et, pour chaque chunk, le score de pertinence et un extrait du texte retrouvé. Cette transparence, inhabituelle dans les assistants conversationnels grand public, est un atout pédagogique important dans le cadre d'un projet académique et renforce la confiance de l'utilisateur dans les réponses fournies.

5.5 Système de suggestions intelligentes

L'interface propose deux niveaux de suggestions. Sur l'écran d'accueil, six cartes affichent une question représentative par démarche, permettant à l'utilisateur de démarrer immédiatement sans avoir à formuler sa propre question. Après chaque réponse, la section 'Vous pourriez aussi demander' propose trois questions de suivi liées à la même démarche, générées localement par détection de mots-clés (sans appel API supplémentaire). Ce mécanisme de suggestion locale est plus fiable que la version précédente basée sur les chunks ChromaDB retournés, car il garantit la cohérence thématique des suggestions quelle que soit la question posée.

6. Déploiement et Infrastructure

Le déploiement de GuichetIA suit une architecture cloud entièrement gratuite, dissociée entre backend et frontend. Cette séparation permet de mettre à jour chaque composant indépendamment et d'optimiser chacun selon ses contraintes propres.

Composant	Plateforme	URL	Spécificités
Backend API	Render (gratuit)	guichetia-backend.onrender.com	512 Mo RAM, mise en veille 15 min
Frontend	Vercel	projet-data-science-guichet-ia.vercel.app	CDN mondial, build automatique
Base vectorielle	ChromaDB (embarqué)	Reconstruit à chaque déploiement	Pas de service externe requis
LLM	Groq API	api.groq.com	Plan gratuit, 6 000 req/min
Embeddings	HuggingFace Inference API	api-inference.huggingface.co	Plan gratuit, modèle multilingue

Tableau 5 : Architecture de déploiement de GuichetIA

6.1 Backend Render

Le backend FastAPI est déployé sur Render via un fichier `render.yaml` versionné dans le dépôt Git. À chaque push sur la branche principale (main), Render déclenche automatiquement un redéploiement : installation des dépendances Python, reconstruction de l'index ChromaDB (`python ingestion/build_index.py`) et démarrage du serveur (`uvicorn app.main:app`). La reconstruction de l'index à chaque déploiement garantit que tout document source mis à jour est immédiatement pris en compte. La limitation principale du plan gratuit est la mise en veille après 15 minutes d'inactivité, ce qui entraîne un délai de démarrage à froid d'environ 30 à 50 secondes pour la première requête après une période d'inactivité.

6.2 Frontend Vercel

Le frontend React est déployé sur Vercel, également connecté au dépôt GitHub. Chaque push déclenche une compilation (`npm run build` via `Vite/rolldown`) et un déploiement instantané sur le CDN mondial de Vercel. L'URL de l'API backend est configurée via la variable d'environnement `VITE_API_URL` définie dans le tableau de bord Vercel. Le frontend bénéficie ainsi de performances optimales (temps de chargement < 1 s) et d'une haute disponibilité.

6.3 Intégration continue

Le projet utilise GitHub comme dépôt central. Chaque commit poussé sur la branche main déclenche automatiquement les pipelines CI/CD de Render (backend) et de Vercel (frontend). Ce mécanisme garantit que la version déployée est toujours synchronisée avec le code source et que toute correction ou amélioration est immédiatement disponible pour les utilisateurs.

7. Tests et Évaluation

L'évaluation de GuichetIA porte sur trois dimensions : la pertinence du retrieval (capacité à retrouver les bons chunks), la qualité des réponses générées (précision, exhaustivité) et les performances techniques (latence, disponibilité).

7.1 Évaluation qualitative

L'évaluation qualitative a porté sur un ensemble de questions représentatives des six démarches couvertes. Pour chaque question, la réponse de GuichetIA a été comparée aux informations officielles disponibles sur les sites gouvernementaux. Les résultats montrent une précision factuelle élevée pour l'ensemble des démarches testées, avec des scores de pertinence ChromaDB cohérents avec la qualité des réponses.

Question de test	Score moyen (%)	Qualité de la réponse
Quelles pièces pour obtenir une CNIB ?	72 %	Excellente : liste complète
Quel est le délai pour un passeport ?	68 %	Bonne : 72 heures ouvrables
Comment créer une entreprise au Burkina ?	65 %	Bonne : étapes CEFORE
Comment obtenir un acte de naissance ?	70 %	Excellente : démarche complète
Quelles pièces pour le casier judiciaire ?	71 %	Excellente : lien e-casier
Comment obtenir un certificat de nationalité ?	69 %	Bonne : pièces et délais

Tableau 6 : Résultats d'évaluation sur un échantillon de questions de test

7.2 Évaluation des performances techniques

Les mesures de performance collectées sur le backend Render (plan gratuit) indiquent une latence totale moyenne de 800 ms à 1 500 ms par requête, dont environ 200 à 500 ms pour la génération LLM (Groq) et 50 à 150 ms pour la recherche vectorielle ChromaDB. Ces performances sont satisfaisantes pour une application web interactive et cohérentes avec les objectifs du projet. La principale limitation reste la latence de démarrage à froid (cold start) de 30 à 50 secondes après une période d'inactivité.

8. Limites et Perspectives

8.1 Limites identifiées

GuichetIA présente plusieurs limites inhérentes à son architecture actuelle. Premièrement, la base documentaire est statique : elle ne se met pas à jour automatiquement lorsque les procédures administratives évoluent et nécessite une intervention manuelle pour régénérer les PDFs sources et reconstruire l'index. Deuxièmement, le modèle d'embedding (paraphrase-multilingual-MiniLM-L12-v2) est généraliste et non spécialisé pour le domaine administratif burkinabè, ce qui peut réduire la pertinence pour des questions très spécifiques. Troisièmement, la mise en veille du backend sur le plan gratuit de Render représente une friction pour l'utilisateur lors du premier accès après inactivité. Quatrièmement, le système ne gère pas le contexte conversationnel : chaque question est traitée indépendamment, sans mémorisation des échanges précédents.

8.2 Perspectives d'évolution

GuichetIA constitue une base solide sur laquelle plusieurs améliorations peuvent être envisagées à court et moyen terme :

- Mise à jour automatique de la base documentaire via scraping périodique des sites officiels.
- Fine-tuning d'un modèle d'embedding spécialisé sur le corpus administratif burkinabè.
- Intégration d'une mémoire conversationnelle pour des échanges multi-tours cohérents.
- Ajout d'un module de synthèse vocale pour rendre l'assistant accessible aux utilisateurs peu alphabétisés ou en situation de mobilité.
- Extension à d'autres démarches (état civil, fiscalité, éducation) et à d'autres pays francophones d'Afrique de l'Ouest.
- Mise en place d'un tableau de bord administrateur pour suivre les questions les plus fréquentes et identifier les lacunes de la base documentaire.

9. Conclusion

GuichetIA démontre qu'il est possible de concevoir et de déployer un assistant administratif intelligent, utile et fiable dans un contexte de contraintes fortes (budget nul, ressources limitées). L'architecture RAG choisie garantit que les réponses sont ancrées dans des sources documentaires officielles et vérifiables, évitant ainsi les hallucinations caractéristiques des LLM utilisés seuls. Le système couvre les six principales démarches administratives auxquelles les citoyens burkinabè sont confrontés et offre une expérience utilisateur intuitive, transparente et informative. Ce projet illustre le potentiel de l'intelligence artificielle appliquée au service public dans les pays en développement, où la fracture informationnelle constitue un obstacle réel à l'exercice des droits des citoyens.

Annexes

Annexe A : Structure du dépôt GitHub

Le projet est organisé en deux répertoires principaux :

- backend/ : code Python du backend FastAPI, scripts d'ingestion, PDFs sources, index ChromaDB
- frontend/ : application React, composants, pages (Landing, ChatDashboard, About), configuration Vite

Annexe B : Variables d'environnement

Les clés API sensibles sont stockées exclusivement en variables d'environnement, jamais dans le code source versionnés :

- GROQ_API_KEY : clé d'accès à l'API Groq pour l'inférence LLM (configurée dans Render)
- HUGGINGFACEHUB_API_TOKEN : clé d'accès à l'API HuggingFace pour les embeddings (configurée dans Render)
- VITE_API_URL : URL du backend FastAPI (configurée dans Vercel)

Annexe C : Commandes de déploiement

La procédure de mise à jour du système suit les étapes suivantes :

15. Mettre à jour les données dans backend/data/generate_source_pdfs.py
16. Régénérer les PDFs : `python data/generate_source_pdfs.py`
17. Commiter et pousser sur GitHub : `git add . && git commit -m 'message' && git push origin main`
18. Render redéploie automatiquement le backend et reconstruit l'index ChromaDB
19. Vercel redéploie automatiquement le frontend